on different nodes or channels, which could lead to load imbalances, especially with small query batches. Such imbalances can be mitigated by larger batches, as it is less likely that all queries happen to hit the same node or channel. Additionally, for the case of uneven access frequencies across IVF lists, adjusting their placement based on these frequencies can help achieve better load balancing [9].

5 IMPLEMENTATION

Chameleon is implemented in 11K lines of code, including 3K lines of Vitis HLS C/C++ for the ChamVS near-memory accelerator, 1.4K lines of C++ for the CPU coordinator, 3.5K lines of Python for ChamLM, and 3.2K lines of Python for various evaluations. Referring to existing RALM research [41, 42], we build ChamLM on Fairseq [60], a PyTorch-based LLM toolkit. ChamLM extends Fairseq to support multi-GPU inference, initiating retrieval requests, integrating the retrieved tokens into generation processes, and network communication between the retrieval engines and GPU processes. ChamVS.idx uses Faiss [40] for index scanning on GPUs or CPUs. ChamVS.mem integrates an FPGA TCP/IP stack [25]. The CPU coordinator process for query broadcasting and result aggregation is implemented in C++ using the socket library. The simple messages in RALMs allow us to avoid higher-level abstractions like RPCs, minimizing performance overhead.

6 EVALUATION

We evaluate Chameleon to answer the following questions:

- How much performance and energy benefits can ChamVS attain in large-scale vector search? § 6.2
- How does Chameleon perform across different RALMs by introducing heterogeneous accelerators? § 6.3
- Is accelerator disaggregation necessary? § 6.3

6.1 Experimental Setup

LLMs. We evaluate RALM models of similar sizes to those in existing RALM research [7, 31, 51, 72, 86], up to several billions of parameters. We evaluate both smaller (S) and larger (L) decoder-only (Dec) and encoder-decoder (EncDec) models. Table 1 summarizes the four RALMs for evaluation, including input dimensionalities, numbers of layers and attention heads, model sizes, retrieval intervals, and neighbor numbers. For EncDec models, we follow [7] to use a two-layer shallow encoder and a deeper decoder, and set different retrieval intervals. For all the models, we use a vocabulary size of 50K and let them generate 512 tokens per sequence.

Vector datasets. Table 2 summarizes the four evaluated vector datasets. The SIFT and Deep datasets are popular benchmarks for billion-scale ANN. Due to the lack of available datasets for RALM, we create two synthetic datasets by replicating each SIFT vector to the models' dimensionalities (512 and 1024). As a common practice, we set *nlist*, the number of clusters in the IVF index, to approximately the square root of the number of dataset vectors (*nlist=32K*). We set *nprobe* as 32 to scan 0.1% of database vectors per query, for which high recall can be achieved on both real-world datasets (93% on Deep and 94% on SIFT for 100 nearest neighbors). We quantize the SIFT and Deep datasets to 16-byte PQ codes, while the two synthetic datasets adopt 32 and 64-byte PQ codes, respectively.

Table 1: Various RALM configurations in the evaluation.

	Dim.	Layers	Heads	Param.	Interval	K
Dec-S	512	24	8	101M	1	100
Dec-L	1024	96	16	1259M	1	100
EncDec-S	512	2,24	8	158M	8/64/512	10
EncDec-L	1024	2,96	16	1738M	8/64/512	10

Table 2: The vector datasets used in the evaluation.

	Deep	SIFT	SYN-512	SYN-1024
#vec	1E+9	1E+9	1E+9	1E+9
m/D	16/96	16/128	32/512	64/1,024
nprobe/nlist	32/32K	32/32K	32/32K	32/32K
Raw vectors (GB)	384	512	2,048	4,096
PQ and vec IDs (GB)	24	24	40	72

Software. For vector search, we use *Faiss* [1] developed by Meta, known for its optimized PQ implementations for both CPUs and GPUs. Due to its vector-only nature, Faiss's ANN search performance surpasses vector data management systems that support additional relational data functionalities [62]. For LLM inference, we extend Fairseq [60] to support RALMs as introduced in §5.

Hardware. We instantiate the ChamVS near-memory accelerator on AMD Alveo U250 FPGAs (16 nm) equipped with 64 GB of DDR4 memory (4 channels x 16 GB) and set the accelerator frequency to 140 MHz. For a fair comparison, each ChamVS memory node is compared to a CPU-based vector search system with equivalent memory capacity (64 GB) and an 8-core AMD EPYC 7313 processor (7 nm) with a base frequency of 3.0 GHz. We evaluate NVIDIA RTX 3090 GPUs (8nm) with 24 GB GDDR6X memory.

6.2 Large-Scale Vector Search on ChamVS

Search performance. We compare ChamVS with baseline systems using four hardware setups. PQ codes can be processed on CPU/FPGA while the IVF index can be scanned on CPU/GPU, leading to four hardware configurations: CPU, CPU-GPU, FPGA-CPU, and FPGA-GPU. To report the best baseline performance, the CPU and CPU-GPU systems are monolithic, while the FPGA-CPU and FPGA-GPU systems are disaggregated over the network. Figure 8 compares the latency distributions of the four solutions. Each white dot in the violin plots denotes a median latency. The number of CPU cores and the number of accelerators used are listed in the plot legends. We make two primary observations from the experiments:

Firstly, the near-memory accelerator in ChamVS significantly lowers vector search latency. Across different datasets and batch sizes (Figure 8), the FPGA-CPU solution achieves 1.36~6.13× speedup compared to the CPU baseline, and the FPGA-GPU solution shows even higher speedup (2.25~23.72×). This is because the ChamVS near memory accelerator can (a) decode PQ codes in parallel, (b) pipeline the decoding, distance calculation, and K-selection, such that each quantized vector can be processed by the pipeline rapidly.

Secondly, scanning the IVF index on GPU allows further latency improvements compared to the FPGA-CPU solution. As shown in Figure 8, the FPGA-GPU approach achieves 1.04~3.87× speedup compared to the FPGA-CPU solution. This is because the IVF index scan procedure can easily leverage the massively parallelism and the high memory bandwidth of GPUs. In contrast, the hybrid CPU-GPU solution shows little or even negative improvements compared